Flutter experiments 6 and 8: learn from zero

These four standalone programs cover all four syllabus parts. Any working code is accepted by your teacher, according to your clarification. The examples retain the required concepts while removing decorative details and repeated app classes. They are based on the supplied syllabus and lab material; they cannot guarantee an examiner's mark.

1. What to open

Part	Code file	What you demonstrate
6(a)	exp6a_custom_widget.dart	A custom `ProfileCard`, reused for two people
6(b)	exp6b_theme.dart	App-wide theme settings and a local text style
8(a)	exp8a_size_animation.dart	A square grows and shrinks using a controller and tween
8(b)	exp8b_fade_slide.dart	Separate squares demonstrate fade and slide; a button replays both

Study in this order: **6(a), 6(b), 8(a), 8(b)**. Each file is an independent program. Do not paste all four into one file: they each contain their own `main()` and `MyPage`.

2. Running a program

In the exam, open the lab's existing Flutter project. Replace the contents of `lib/main.dart` with ONE complete program and run it. To change experiments, replace the whole file and stop/restart the app. This also reruns animation initialization.

On your Mac, your current project is `first_flutter/flutter_application_1`. The new files are already inside its `lib/exam_prep_6_8` folder. From that project's terminal, you can run a file without replacing your existing main.dart:

```
flutter run -d chrome -t lib/exam_prep_6_8/exp6a_custom_widget.dart
```

Change the last filename for the other programs. These programs need only Flutter's built-in material library: no images, downloads, extra packages, or internet requests at runtime. A configured Flutter project and SDK are still required. The same Dart code works on Windows; the local folder path differs.

3. The foundation: Flutter builds a screen from widgets

Dart is the language. **Flutter** supplies the widgets used to create the screen. A **widget** describes a piece of the user interface, such as text, a button, a card, or a layout.

Imagine assembling a screen from boxes inside boxes:

```
MaterialApp                App settings and Material support
  MyPage                   Our screen
    Scaffold               Basic screen layout
      appBar: AppBar       Top title bar
      body: Column         Main content arranged vertically
        ProfileCard        First person's card
        ProfileCard        Second person's card
```

`child:` holds one widget. `children: [...]` holds a list of widgets. `body:` is the Scaffold's main content area. `title:` names the content used as a title. These are named arguments: the word before the colon says which setting you are supplying.

Read the common opening

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(home: MyPage()));
}
```

- `import` makes Flutter's Material widgets available.
- `main()` is where execution begins.
- `void` means the function does not return a result.
- `runApp(...)` starts displaying the supplied widget.
- `MaterialApp(...)` provides app-level Material support and settings.
- `home: MyPage()` makes our `MyPage` widget the initial screen.

An object is a particular instance created from a class. `MyPage` is a class; `MyPage()` creates an instance of it. You do not need a separate `MyApp` class just to wrap `MaterialApp`, so these examples omit it.

Symbols you will keep seeing

Syntax	Meaning here
`class MyPage`	Defines our widget type
`extends StatelessWidget`	Uses Flutter's base for a widget without its own mutable State object
`@override`	We are providing our implementation of an inherited method
`Widget build(BuildContext context)`	A method that returns the widgets to display
`context`	Identifies this widget's location in the widget tree
`return`	Sends the result back to the caller
`()`	Function/constructor arguments, or method parameters
`{}`	A class/function body; inside parameters, also used for named parameters
`[]`	A list, such as a list of child widgets
`;`	Ends a statement
`const`	Creates a constant object when the supplied values allow it
`super.key`	Passes an optional widget identity key to Flutter's parent constructor

Keep `const MyPage({super.key});` as the standard constructor line. You do not need to invent a key for these exercises. A StatelessWidget can still rebuild when its parent or inherited settings change; it simply does not manage a separate mutable State object of its own.

4. Experiment 6(a): custom widgets

Expected output: two cards, one showing Rahul / Student and another showing Krishna / Teacher.

Open `exp6a_custom_widget.dart`. It has two classes:

1. `MyPage` creates the screen and uses the cards.
2. `ProfileCard` defines how any one person's card looks.

The screen

`Scaffold` creates a screen with a title bar and a body. The `Column` stacks its children vertically. These two lines create two instances of the same custom widget:

```
ProfileCard(name: 'Rahul', role: 'Student'),
ProfileCard(name: 'Krishna', role: 'Teacher'),
```

The layout is reused; only the input values change. This is the central requirement of 6(a).

The information each card stores

```
final String name;
final String role;

const ProfileCard({super.key, required this.name, required this.role});
```

- `String` means text.
- `name` and `role` are fields: information stored in each card object.
- `final` means each field is assigned once for that object.
- The constructor has the same name as its class: `ProfileCard`.
- `required` means the caller must supply that argument.
- `this.name` stores the supplied name in this card's `name` field. `this.role` does the same for the role.

For the first card, `name` is `'Rahul'` and `role` is `'Student'`. For the second card, the fields hold different values.

The card's appearance

```
return Card(
  child: ListTile(
    leading: const Icon(Icons.person),
    title: Text(name),
    subtitle: Text(role),
  ),
);
```

`Card` supplies the card surface. `ListTile` gives a ready-made layout: an icon on the left, a main title, and a subtitle. `Text(name)` displays the variable's value. `Text('name')` would display the literal word **name**, so do not put quotes around the variable.

Why this is sufficient: it defines an actual custom widget class, accepts inputs, and reuses it. An avatar, arrow, shadow setting, and extra spacing are decoration, not the custom-widget concept.

Practise: change Rahul to your own name. Then add a third `ProfileCard` inside `children`. Finally, close the code and recreate it in this order: import and main, page, card fields, constructor, card build method.

Viva answer: "A custom widget is a reusable UI component created by the developer. My ProfileCard accepts a name and role, and I reuse it with different values."

5. Experiment 6(b): themes and custom styles

Expected output: a pale yellow screen, a blue title bar with white text, a bold 28-size Welcome heading, and red 20-size Flutter Lab text.

Open `exp6b_theme.dart`. The screen structure is familiar. The new idea is defining common appearance settings inside `MaterialApp`:

```
theme: ThemeData(
  scaffoldBackgroundColor: Colors.yellow.shade100,
  appBarTheme: const AppBarTheme(
    backgroundColor: Colors.blue,
    foregroundColor: Colors.white,
  ),
  textTheme: const TextTheme(
    headlineSmall: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
  ),
),
```

Think of `ThemeData` as the app's shared appearance settings:

- `scaffoldBackgroundColor` sets the default screen background.
- `appBarTheme` sets the default appearance of title bars.
- `textTheme` defines named text styles.
- `headlineSmall` is one of Flutter's predefined style slots; we choose its font size and weight.
- `shade100` selects a light shade of yellow.

Defining a named text style does not make every Text widget use that slot. We select it for Welcome:

```
Text('Welcome', style: Theme.of(context).textTheme.headlineSmall)
```

Read that from left to right: find the theme above this screen, access its text styles, and use `headlineSmall`.

For Flutter Lab, we apply a custom style directly:

```
Text('Flutter Lab', style: TextStyle(color: Colors.red, fontSize: 20))
```

This affects that particular text. The example therefore clearly demonstrates both **theme settings** and **a local custom style**.

`Center` centers the content. `Column` arranges the two text widgets vertically. `mainAxisSize: MainAxisSize.min` asks the Column to occupy only the vertical space its children need, allowing Center to center that group.

Correction to the supplied sample: its `Theme.of(context)` uses the context of the widget creating MaterialApp, which is above the new theme. Here `MyPage` is inside MaterialApp, so its context can find the theme correctly.

Practise: change the theme background to green.shade100 and Welcome's size to 32. Change only the local text color to purple. Notice which settings affect which widget.

Viva answer: "A theme defines shared visual settings for the app. A custom style changes the appearance of an individual widget."

6. Experiment 8(a): size animation

Expected output: a blue 50-by-50 square and an Animate button. Press the button: the square grows to 200-by-200 over two seconds. After it finishes, press again: it shrinks back.

First understand what an animation does

The square does not jump directly from 50 to 200. Its size passes through intermediate values:

```
Time:               0 seconds      1 second       2 seconds
Controller value:   0              0.5            1
Square size:        50             125            200
```

The **controller** manages progress and timing. A **tween** maps that progress to our chosen start and end values. Rebuilding displays the new size.

Why two classes now?

```
class MyPage extends StatefulWidget {
  const MyPage({super.key});

  @override
  State<MyPage> createState() => _MyPageState();
}
```

`MyPage` is now stateful because this example needs an object that manages animation state and its lifetime. `createState()` creates that object. `=> expression` is a short form for `{ return expression; }`.

```
class _MyPageState extends State<MyPage>
    with SingleTickerProviderStateMixin {
```

`State<MyPage>` means this state belongs to MyPage. An initial underscore makes the class private to its Dart library. The mixin gives this State the ability to supply a ticker: Flutter's frame-timing signal for the controller. One controller is enough here.

Declare the two objects

```
late AnimationController controller;
late Animation<double> size;
```

`late` means we will initialize the field before using it. `Animation<double>` supplies changing decimal-number values. We call it `size` because its current value will determine the square's width and height.

Initialize once

`initState()` runs once when this State is inserted into the tree. `super.initState()` runs the parent initialization first.

```
controller = AnimationController(
  vsync: this,
  duration: const Duration(seconds: 2),
);
size = Tween<double>(begin: 50, end: 200).animate(controller);
controller.addListener(() {
  setState(() {});
});
```

- `duration` specifies the forward animation's length.
- `vsync: this` uses the ticker provider supplied by this State's mixin.
- `Tween<double>` describes the 50-to-200 range.
- `.animate(controller)` connects that range to the controller's progress.
- `addListener` registers a function to run when the controller's value changes.
- `() { ... }` is an unnamed function, also called an anonymous function or callback.
- `setState` asks Flutter to rebuild this State's UI. The callback is empty here because the controller has already changed the value we read.

Do not create the controller in `build()`: build can run repeatedly. Create it in `initState()` and clean it up in `dispose()`.

Display the current value

```
Container(width: size.value, height: size.value, color: Colors.blue)
```

`size.value` is the current animated number. The width and height use the same number, so the shape stays square.

Start or reverse

```
if (controller.isCompleted) {
  controller.reverse();
} else {
  controller.forward();
}
```

This callback runs when the button is pressed. `isCompleted` is true when progress has reached the upper end. `reverse()` goes toward the starting value; `forward()` goes toward the ending value. For the demonstration, let each animation finish before pressing again.

Clean up

```
@override
void dispose() {
  controller.dispose();
  super.dispose();
}
```

When the State is removed permanently, Flutter calls dispose. We release the controller's resources, then call the parent's cleanup method. Keep this block.

Memory order: declare objects, initialize them, build the display, dispose the controller.

Practise: change the end size to 150 and the duration to four seconds. Predict the size halfway through: `(50 + 150) / 2 = 100`.

Viva answer: "AnimationController controls timing and progress. Tween maps progress to a start and end value. I rebuild the square using the current animated size."

7. Experiment 8(b): fade and slide

Expected output: a blue square gradually appears and a red square moves right into its normal position. Both start automatically and finish after two seconds. Replay starts both again.

Open `exp8b_fade_slide.dart`. Its main function, StatefulWidget, State class, controller initialization, and disposal follow the same structure as 8(a).

Fade: change visibility

```
FadeTransition(
  opacity: controller,
  child: Container(width: 100, height: 100, color: Colors.blue),
)
```

Opacity means how visible something is: 0 is invisible, 0.5 is partly transparent, and 1 is fully visible. An AnimationController is itself an Animation and defaults to the range 0 to 1. It can therefore be supplied directly as opacity. A separate 0-to-1 tween would produce the same values and is unnecessary here.

An invisible square still occupies layout space. Fade changes visibility, not the space reserved for the widget.

Slide: change position

```
late Animation<Offset> position;
```

`Offset` holds two numbers: horizontal and vertical displacement. We initialize the animation with:

```
position = Tween<Offset>(
  begin: const Offset(-1, 0),
  end: Offset.zero,
).animate(controller);
```

In SlideTransition, offsets are measured relative to the child's own size:

- `Offset(-1, 0)`: one full child-width to the left, with no vertical shift.
- `Offset.zero`: the normal position, equivalent to `Offset(0, 0)`.
- For a 100-wide square, the horizontal displacement changes from -100 to 0 logical pixels.
- It is not necessarily outside the screen at the start; it is left of its normal position.

```
SlideTransition(
  position: position,
  child: Container(width: 100, height: 100, color: Colors.red),
)
```

The first `position` is the widget's argument name. The second is our animation variable.

Why no addListener or setState here?

FadeTransition and SlideTransition listen to the animation and update their display automatically. In 8(a), we directly read a number into an ordinary Container, so we explicitly requested rebuilds.

Both transitions share one controller because they use the same duration and progress. This avoids writing two entire apps while keeping the effects separate and easy to demonstrate.

`controller.forward()` at the end of initState starts the animation automatically. The Replay button calls `controller.forward(from: 0)` to reset progress to zero and start again.

Practise: change `Offset(-1, 0)` to `Offset(0, -1)`. The red square will start one square-height above its normal position and slide down into place.

Viva answer: "FadeTransition changes opacity. SlideTransition changes position. Both can use the same controller, and the transition widgets update themselves when it changes."

If the examiner specifically asks for scale

The supplied PDF also includes scale. The syllabus names fade and slide as examples, which the main program covers. To demonstrate scale as an additional type, add this widget inside the same Column's children (reduce square sizes if the screen becomes crowded):

```
ScaleTransition(
  scale: controller,
  child: const FlutterLogo(size: 60),
),
```

The logo visually scales from 0 to its full size, using the same controller. To make a fade-only or slide-only program, retain the matching Transition and remove the other one and its label. For fade-only,

also remove the unused `position` declaration and its Tween initialization. Keep the controller and disposal.

8. How to practise without memorizing blindly

For each program:

1. Read the expected output before reading the code.
2. Run it and see whether the output matches.
3. Explain each block aloud using the guide.
4. Change one value and predict what will happen before running.
5. Close the reference and type the program yourself.
6. Compare and fix the blocks you forgot. Repeat once.

Learn the common screen structure first. Then remember the unique requirement:

Part	The block you must be able to explain
6(a)	ProfileCard class, its constructor, and reuse with different values
6(b)	ThemeData, Theme.of(context), and a direct TextStyle
8(a)	Controller, Tween, listener/setState, and current size
8(b)	Opacity 0 to 1, Tween, and the two Transition widgets

Common mistakes: missing commas between named arguments; missing `required` values; quoting variable names; putting two programs in one file; forgetting the ticker mixin, initState, or dispose; expecting hot reload to rerun initState. Use a full restart when switching examples.

References

Local requirements: `Flutter_syllabus.pdf`, experiment 6 text files, and `exp8/Flutter-labsession-8.pdf` in the Flutter course folder.

- Flutter: Theme.of and the location of BuildContext
- Flutter: AnimationController and its lifecycle
- Flutter: SlideTransition offsets
- Flutter: animations overview